

1.野指针？

原因：在声明指针时将其初始化为NULL：当我们声明一个指针但没有为其分配有效的内存地址时，可以将其初始化为NULL，表示该指针不指向任何有效的内存位置；在释放内存后未将指针置为NULL：当我们释放了一个指针所指向的内存后，为了避免野指针的产生，应该将指针设置为NULL。否则，该指针将继续指向已经释放的内存位置，成为野指针。

危害：可能会导致程序崩溃，内存泄漏或者会产生无法预测的行为（进行解引用操作）

避免方法：首先要初始化指针，并判断指针有效性，或者使用智能指针，或者异常处理，在释放动态分配的内存后，将指针置空，可以避免再次使用已释放的内存。在使用指针进行数组操作时，确保访问的索引在有效范围内。

2.IP协议是可靠的吗

IP 协议属于 TCP/IP 体系中的网络层协议，其核心功能是实现数据包的路由和转发，但不提供可靠性保障：

不保证数据包的顺序交付（可能乱序）；
不保证数据包的完整性（可能丢失或损坏）；
没有重传机制（丢失的数据包不会主动重传）；
没有拥塞控制（可能因网络拥堵导致数据包丢弃）。

可靠性通常由上层协议（如传输层的 TCP）通过确认应答、超时重传、序号校验等机制实现。

3.this指针的作用

this 指针是 C++ 中类的成员函数里隐含的指针，指向当前调用该成员函数的对象实例，其主要作用包括：

区分成员变量与局部变量：当成员变量与函数参数 / 局部变量同名时，通过this->成员变量明确访问对象的成员变量（如this->name = name）。
在成员函数中访问当前对象的地址：可用于返回对象自身（如链式调用：return *this），或作为参数传递给其他函数。
隐含访问成员：成员函数中访问成员变量 / 函数时，编译器会自动通过 this 指针实现（如name实际等价于this->name）。

4.索引失效

索引失效指查询本应使用索引却未使用（转而全表扫描），常见原因包括：

索引列参与函数 / 表达式运算：如WHERE SUBSTR(name, 1, 3) = 'abc'、WHERE age + 1 = 20，索引无法直接匹配运算后的值。
使用不等判断（!=、<>、NOT IN、IS NOT NULL）：这类操作可能导致索引失效（视数据库优化器而定）。
字符串不加引号：如WHERE name = 123（实际应为字符串），导致类型转换，索引失效。
联合索引不满足“最左前缀原则”：如联合索引(a, b, c)，仅查询b或c时，索引不生效。
OR 连接的条件中存在无索引列：如WHERE a = 1 OR b = 2，若b无索引，可能导致整个查询不使用索引。
LIKE 以“%”开头：如WHERE name LIKE '%abc'，无法利用索引的前缀匹配特性。
索引列被更新 / 删除导致碎片化严重：极端情况下，数据库可能认为全表扫描比使用碎片化索引更高效。

5.键入URL的过程

在浏览器中输入指定网页的 URL。

浏览器通过 DNS 协议，获取域名对应的 IP 地址。

浏览器根据 IP 地址和端口号，向目标服务器发起一个 TCP 连接请求。

浏览器在 TCP 连接上，向服务器发送一个 HTTP 请求报文，请求获取网页的内容。

服务器收到 HTTP 请求报文后，处理请求，并返回 HTTP 响应报文给浏览器。

浏览器收到 HTTP 响应报文后，解析响应体中的 HTML 代码，渲染网页的结构和样式，同时根据 HTML 中的其他资源的 URL（如图片、CSS、JS 等），再次发起 HTTP 请求，获取这些资源的内容，直到网页完全加载显示。

浏览器在不需要和服务器通信时，可以主动关闭 TCP 连接，或者等待服务器的关闭请求。

6.C++构造函数可以私有的，怎么创建这个类？

私有构造函数通常用于控制类的实例化方式（例如，实现单例模式、工厂模式、禁止栈上分配、只允许友元或静态成员函数创建等）。

可以通过在类内部定义一个`public static`成员函数（常命名为`createInstance()`、`getInstance()`等）。这个函数内部可以调用私有构造函数（因为它在类作用域内），并返回（通常是堆上分配的）对象指针或引用。

或者将外部类设置成友元类进行使用。

7. 拷贝构造函数使用的注意事项

1). 注意深拷贝和浅拷贝问题：如果类内有动态内存或者其他资源，需要用深拷贝，否则调用析构函数时会出现多次释放空间或者悬空指针。

2). 参数中必须加入const进行引用

- 引用：避免无限递归（如果传值，为了传值需要调用拷贝构造，为了调用拷贝构造又需要传值...）。

- `const`：避免意外修改源对象，也允许用临时对象或常量对象作为源进行拷贝。

3). 还要有自我赋值的判断

4). 调用基类拷贝构造：如果类继承自基类，必须在派生类拷贝构造的初始化列表中显式调用基类的拷贝构造函数，否则编译器会调用基类的默认构造函数（通常不是你想要的）。

8. 虚函数和纯虚函数的区别？

虚函数 允许在派生类中重写函数，但可以有默认实现。它是可选的，可以在基类中提供实现。

纯虚函数 没有默认实现，派生类必须提供实现。它使得基类成为抽象类，不能实例化。

类如果声明了虚函数，这个函数是实现了的，即使是空实现，它的作用就是为了让这个函数在它的子类里面可以被覆盖，这样编译器就可以使用动态绑定来达到多态的目的（即父类指针指向子类对象，调用子类方法）。而纯虚函数只是在基类中的一个函数定义，即是一个函数声明而已，具体的实现需要留到子类当中。

虚函数在子类里面也可以不进行重写（只有虚方法和抽象方法才能够被重写）；但纯虚函数必须在子类去实现。

虚函数的类用于“实作继承”，也就是说继承接口的同时也继承了父类的实现。当然，子类也可以进行覆写，从而完成自己关于此函数的实现。纯虚函数的类用于“介面继承”，即纯虚函数关注的是接口的统一性，实现由子类去完成。

带纯虚函数的类叫做抽象类，这种类不能直接生成对象，而只有被继承，并重写其虚函数后，才能使用。

9. 事务的隔离级别

回答：在 MySQL 中，事务隔离级别有四种，读未提交，读已提交，可重复读，串行化，分别隔离等级从低到高越来越严格，隔离等级越高，数据一致性就越好，但性能开销就大。

读未提交

含义：在这个隔离级别下，A 事务中可以看到其他事务未提交的数据；

等级：隔离级别最低；

问题：好家伙事务就是为了 ACID 特性，你直接读到别人没有提交的数据，岂不是虚晃一枪，没确定的事情就不要乱散布谣言，以讹传讹；基本上不会用的

影响：读到别人没提交的数据也就是脏读

读已提交

含义：该事务隔离级别下，当前事务只能读取其他事务已经提交的数据

等级：隔离级别稍微高一些，至少解决了脏读

问题：虽然是读取的已经提交的事务，但是比如某个事务需要开始读某个记录，但是被另一个事务修改了这个记录并且成功提交，那么再次读取这个记录就会造成前后数据不一致的情况，就会有点迷惑；

影响：事务内，前后读取同一条记录的结果不同，也就是不可重复读

可重复读

含义：默认的事务隔离级别，保证在同一个事务内，多次读取数据集会得到相同的结果，及时其他事务在这个中间提交了修改更新

等级：既保证性能，又保证可以重复读，就是好用哇，解决了脏读、不可重复读

问题：虽然是默认隔离等级，但就没有问题吗，？？？大写的问号，如在同一事务内执行两次相同的查询，结果行数不同，就是有可能中间有其他事务突然修改插了一脚，真是凌门一脚踢出来个 MVCC 解决方案，不然咋并发

影响：那就是有可能造成幻读，幻读，迷幻哇，不就是迷惑了，比如在同一事务中，原本读我的邮箱有 100 条记录，又读了一次变成 200 条了，就有点晕了，跟喝酒看重影一样

串行化

含义：串行化，顾名思义，通过强制事务串行执行，每次事务会在读取的每一行数据上加锁，确保其他事务不能同时进行修改、删除或插入操作

等级：最最最严格的等级

问题：效率低，事务变成串行了，数据库的并发性能就低下，而且引入了锁，对资源的消耗也上来了，说不定还有死锁的风险

影响：数据一致性等级最高，但性能很低，基本上也不咋用。

10. 编码流程

初始化阶段（slot_setInfo）中，首先创建AVFormatContext并指定输出格式为FLV（备选MPEG）；接着按配置添加音视频流并配置编码器，视频流通过add_vedio_stream添加H.264编码流（AV_CODEC_ID_H264），配置比特率1,000,000 bps、分辨率取自桌面尺寸、帧率25 fps（time_base=1/FRAME_RATE）、像素格式YUV420P、GOP大小12，音频流通过add_audio_stream添加AAC编码流（AV_CODEC_ID_AAC），配置采样率44,100 Hz、声道布局AV_CH_LAYOUT_STEREO、采样格式AV_SAMPLE_FMT_FLTP、比特率64,000 bps；同时进行实时编码优化，视频编码开启zerolatency和superfast预设，线程数设为4；随后通过avcodec_open2初始化编码器，视频分配AVFrame并绑定YUV420P缓冲区，音频分配AVFrame并绑定FLTP缓冲区且计算每帧大小；数据采集与转换方面，视频通过PicInPic_Read捕获摄像头/桌面画面，转RGB24后用sws_scale转为YUV420P，音频通过QAudioInput采集PCM S16，用swr_convert转为FLTP；数据队列管理借助QMutex保障线程安全，视频队列基于时间戳淘汰过期帧，音频队列采用FIFO管理；编码与封装在run循环中，用av_compare_ts同步音视频，优先处理时间戳较小的流，将队列数据复制到缓冲区并设置PTS，调用avcodec_send_frame和avcodec_receive_packet编码，经av_packet_rescale_ts转换时间基后用av_interleaved_write_frame写入文件；终止时，在slot_closeVedio中清空队列编码剩余帧，用av_write_trailer写文件尾，关闭编码器并释放帧、缓冲区、输出文件及格式上下文。

11. 为什么用H264，不用H265？

H265的压缩效率比H264提升约50%，有着更高的分辨率或更细腻的画质，相同画质下H265编码的视频文件大小可能是H264的一半，H265采用了更先进的编码算法。

但是H264发布于2003年，几乎所有的设备都原生支持，兼容性更好，H265（2013年发布）普及时间较短，部分设备不支持H265。H264是小流量场景是免费的，H265授权复杂。

12. 介绍H264？

将每一个画面都划分成若干16*16的宏块，之后采用帧内预测编码和帧间预测编码来实现，帧内预测编码用来缩减图像的空间冗余，在对一给定宏块编码时，首先可以根据周围的宏块预测（典型的是根据左上角宏块、左边宏块和上面宏块，因为此宏块已经被编码处理），然后对预测值与实际值的差值进行编码；帧间预测编码利用连续帧中的时间冗余来进行运动估计和补偿，使用两帧之间的运动矢量进行预测，第三步是计算原始宏块和预测宏块之间做一个残差，第四步就是进一步去除里面的一些不重要的信息，最后将所有的信息进行熵编码（通用可变长编码（UVLC）和基于文本的自适应二进制算术编码（CABAC））转换成H264格式的编码。